

REPASO INTEGRADOR · PARTE 2 DE 3

Programación Orientada a Objetos en C++

Repaso Pre-Parcial

Caso de estudio: Sistema de Biblioteca — Módulos 4 a 7

```
class ObjetoDeEstudio {  
private:  
    Conocimiento conocimiento;  
public:  
    void prepararParcial();  
};
```

Herencia y Polimorfismo · Sobrecarga de Operadores · Memoria Dinámica · Copia de Objetos

Programación Orientada a Objetos - Leonardo Brambilla - FCyT - UADER

MÓDULO

4

≈ 35 minutos

Herencia y Polimorfismo

Objetivos de esta sección

- Implementar herencia con virtual y override.
- Diseñar clases abstractas con métodos virtuales puros.
- Explicar el enlace dinámico y predecir qué método se ejecuta.

Herencia: sintaxis, virtual y override

- ▶ `class Socio : public Persona {}` hace que Socio herede atributos y métodos públicos/protegidos de Persona.
- ▶ **virtual** en la clase base habilita el enlace dinámico: la llamada se resuelve según el tipo real del objeto, no del puntero.
- ▶ **override** (desde C++11) le indica al compilador que este método debe sobrescribir el virtual de la clase base; detecta errores de firma.
- ▶ Sin virtual, C++ usa enlace estático: se ejecuta la versión de la clase del puntero/referencia, no la del objeto real.

persona.h

```
class Persona {
protected:
    string nombre;
public:
    Persona(string n) : nombre(n) {}
    virtual void presentarse() const {
        cout << "Soy " << nombre;
    }
};

class Socio : public Persona {
public:
    Socio(string n) : Persona(n) {}
    void presentarse() const override {
        cout << "Socio: " << nombre;
    }
};
```

⚠ **Error frecuente:** Olvidar 'virtual' en la clase base: el compilador no avisa, pero el polimorfismo deja de funcionar silenciosamente.

? ¿Qué pasaría si Socio hereda de Persona con 'protected' o 'private' en vez de 'public'?

Clases abstractas y métodos virtuales puros

- ▶ Un **método virtual puro** se declara con '= 0' y no tiene implementación en la clase base.
- ▶ Una clase con al menos un método virtual puro es **abstracta**: no se pueden crear objetos de ella directamente.
- ▶ Las clases derivadas deben implementar todos los métodos puros para poder instanciarse.
- ▶ Sirve para definir un '**contrato**' común sin imponer una implementación por defecto.

publicacion.h

```
class Publicacion {
public:
    virtual int diasDePrestamo() const = 0;
    virtual ~Publicacion() {}
};
// Publicacion p;    // ERROR: es abstracta

class Libro : public Publicacion {
public:
    int diasDePrestamo() const override {
        return 14;
    }
};

class Revista : public Publicacion {
public:
    int diasDePrestamo() const override {
        return 7;
    }
};
```

 **Error frecuente:** Intentar instanciar directamente una clase abstracta, o dejar sin implementar algún método puro en la clase derivada (también queda abstracta).

 ¿Por qué el destructor de *Publicacion* también debería ser virtual? (pista: se retoma en Módulo 6-7 con memoria dinámica).

Polimorfismo y enlace dinámico

- ▶ **Polimorfismo**: un mismo mensaje (llamada a método) produce comportamientos distintos según el tipo real del objeto.
- ▶ Se logra usando punteros o referencias a la clase base que apuntan a objetos derivados.
- ▶ Enlace dinámico (**dynamic binding**): la decisión de qué método ejecutar se toma en tiempo de ejecución, consultando la tabla virtual (**vtable**) del objeto.
- ▶ Sin virtual, la decisión se toma en tiempo de compilación (**enlace estático**) según el tipo del puntero.

Enlace dinámico

```
Persona* p = new Bibliotecario("Marta");  
p->presentarse();
```

```
p (tipo Persona*)  
|  
v  
[ objeto real: Bibliotecario ]  
[ vtable -> Bibliotecario::presentarse ]
```

Se ejecuta `Bibliotecario::presentarse()`
aunque 'p' sea de tipo `Persona*`

? ¿Qué se imprime si `Persona::presentarse` NO es virtual y hacemos `'Persona* p = new Bibliotecario("Marta"); p->presentarse();'`?

Comparación: con virtual vs. sin virtual

- ▶ Ambos fragmentos compilan sin errores: la diferencia es sólo el resultado en tiempo de ejecución.
- ▶ Con virtual, el sistema recorre la vtable del objeto real (Socio).
- ▶ Sin virtual, se resuelve según el tipo declarado del puntero (Persona) en tiempo de compilación.
- ▶ Este es un error frecuente.

CON virtual → "Socio"

```
class Persona { public:  
    virtual string rol(){  
        return "Persona";} };  
class Socio:public Persona{  
    public: string rol() {  
        return "Socio";} };  
Persona* p=new Socio();  
cout<<p->rol(); //Socio
```

SIN virtual → "Persona"

```
class Persona { public:  
    string rol(){  
        return "Persona";} };  
class Socio:public Persona{  
    public: string rol(){  
        return "Socio";} };  
Persona* p=new Socio();  
cout<<p->rol(); //Persona
```

? En el sistema de Biblioteca, ¿qué método de Persona necesitaría ser virtual para que Bibliotecario y Socio se comporten polimórficamente?

Ejercicio: qué método se ejecuta?

- ▶ Analizá el fragmento de la derecha línea por línea antes de responder.
- ▶ Prestá atención a si el método está marcado virtual y a través de qué tipo de puntero se lo invoca.
- ▶ Este patrón de pregunta ('¿qué imprime?', '¿qué método se ejecuta?') aparece con frecuencia en el parcial.

¿Qué imprime este programa?

```
class Publicacion {
public:
    virtual void identificar() {
        cout << "Publicacion"; }
    void tipo() { cout << "tipo-Public"; }
};
class Libro : public Publicacion {
public:
    void identificar() override {
        cout << "Libro"; }
    void tipo() { cout << "tipo-Libro"; }
};
int main() {
    Publicacion* p = new Libro();
    p->identificar();    // ?
    p->tipo();           // ?
}

Libro
tipo-Public
```

? ¿Qué imprime `p->identificar()` y qué imprime `p->tipo()`? Justificá la diferencia entre ambas llamadas.

Resumen — Herencia y Polimorfismo

✓ Ideas clave

- Herencia modela ES-UN; virtual + override habilitan el enlace dinámico.
- Una clase con un método virtual puro (=0) es abstracta y no se puede instanciar.
- Polimorfismo: el método ejecutado depende del tipo real del objeto, no del puntero, sólo si es virtual.
- Sin virtual, se aplica enlace estático según el tipo declarado del puntero/referencia.

⚠ Errores frecuentes

- Olvidar 'virtual' en la clase base y perder el polimorfismo silenciosamente.
- Dejar sin implementar un método puro en la derivada (sigue siendo abstracta).
- Destructor de la clase base no-virtual en una jerarquía polimórfica.
- Confundir override (marca la intención) con la palabra que realmente habilita el polimorfismo (virtual en la base).

📝 Ejercicio rápido

Agregá a la jerarquía Publicacion una clase DVD con diasDePrestamo() propio, creá un vector dinámico Publicacion** con un Libro y una Revista, y recorré llamando a diasDePrestamo() de forma polimórfica.

MÓDULO

5

≈ 20 minutos

Sobrecarga de Operadores

Objetivos de esta sección

- Sobrecargar operadores aritméticos y de comparación.
- Distinguir cuándo implementar un operador como método y cuándo como función friend.

Operadores sobrecargables: contexto

- ▶ La **sobrecarga de operadores** permite que objetos de una clase usen la sintaxis de operadores del lenguaje (+, ==, <<, etc.).
- ▶ **friend** otorga a una función externa acceso a los miembros privados de la clase, sin ser un método de la clase.
- ▶ Regla práctica:
 - Si el operando izquierdo es un objeto de la clase → normalmente implementarlo como método.
 - Si el operando izquierdo pertenece a otra clase (ostream, istream, etc.) → implementarlo como función no miembro. Se necesita friend.

Operadores comunes

Operador	Uso típico	Implementación habitual
operator+	suma de dos objetos	método o friend
operator-	resta de dos objetos	método o friend
operator==	comparar igualdad	método (const)
operator=	asignación (Módulo 7)	método
operator<<	imprimir con cout	friend (obligatorio)
operator>>	leer con cin	friend (obligatorio)



De este tema pueden tener machete, no hace falta que se lo memoricen.

? ¿Por qué operator<< no puede implementarse como método de la propia clase?

Ejemplo completo: clase Fecha (vencimiento de préstamo)

- ▶ **operator+** como método miembro: suma días a una fecha de vencimiento (this siempre es una Fecha).
- ▶ **operator==** como método const: compara si dos fechas son iguales, sin modificar ninguna.
- ▶ **operator<<** como función friend: necesita acceso a los atributos privados pero el primer operando es ostream.

fecha.h

```
class Fecha {
private:
    int dia, mes, anio;
public:
    Fecha(int d=1,int m=1,int a=2026)
        : dia(d), mes(m), anio(a) {}
    Fecha operator+(int dias) const {
        return Fecha(dia+dias, mes, anio);
    }
    bool operator==(const Fecha& o) const {
        return dia==o.dia && mes==o.mes
            && anio==o.anio;
    }
    friend ostream& operator<<(ostream& os,
                               const Fecha& f) {
        os<<f.dia<<"/"<<f.mes<<"/"<<f.anio;
        return os;
    }
};
```

 **Error frecuente:** Olvidar declarar operator== o operator+ como const, impidiendo usarlos sobre objetos const o temporales.

? ¿Qué debería devolver operator<< y por qué es importante que devuelva ostream&?

Resumen — Sobrecarga de Operadores

✓ Ideas clave

- Sobrecargar operadores aplica sintaxis natural a operaciones entre objetos.
 - **Método miembro** cuando el operando izquierdo siempre es del tipo de la clase.
 - **friend** cuando se necesita acceso privado pero el operando izquierdo no es la clase (`operator<<`, `operator>>`).

⚠ Errores frecuentes

- Implementar `operator<<` como método miembro (no compila con la sintaxis `cout << obj`).
- Olvidar 'const' en operadores que no modifican el objeto.
- No devolver la referencia adecuada, rompiendo el encadenamiento de operadores.

📝 Ejercicio rápido

Agregá a la clase `Fecha` un `operator-` (que calcule la diferencia en días entre dos fechas, versión simplificada) y un `operator>>` como función `friend` para leer día, mes y año desde `cin`.

MÓDULO

6

≈ 35 minutos

Memoria Dinámica y Vectores Dinámicos

Objetivos de esta sección

- Diferenciar stack y heap; usar new, delete y delete[] correctamente.
- Manipular punteros a objetos.
- Crear, modificar, redimensionar y liberar vectores dinámicos paso a paso.

Stack vs. Heap, new / delete / delete[]

- ▶ En C++, distinguiremos principalmente dos regiones de memoria:
 - **Stack (pila)**: memoria automática, administrada por el compilador.
 - **Heap (montículo)**: memoria dinámica, administrada por el programador

- ▶ **Stack**: memoria automática, se libera sola al salir de alcance (variables locales, objetos por valor).
- ▶ **Heap**: memoria dinámica, solicitada con new y liberada manualmente con delete; persiste hasta que se libera explícitamente.
- ▶ **delete[]** debe usarse cuando se reservó con new[] (arreglo); usar delete simple sobre un arreglo es comportamiento indefinido.
- ▶ **Memory Leak**: memoria del heap reservada con new que nunca se libera con delete, quedando inaccesible pero ocupada.

Stack vs. Heap

```
STACK (automática)      HEAP (dinámica)
+-----+               +-----+
| Libro l1;             | new Libro(...) |
| (se libera sola      | (delete manual) |
| al salir de          |                 |
| {} )                  | Libro* p =
+-----+               | new Libro();
                        | delete p;
                        +-----+
```

 **Error frecuente:** Usar delete p; sobre un arreglo reservado con new Libro[10]; debe ser delete[] p;.

 Si hago 'Libro* p = new Libro("X");' y la función termina sin delete p;, ¿qué ocurre con esa memoria?

Punteros a objetos

- ▶ Un puntero a objeto almacena la dirección de memoria de un objeto.
- ▶ Se accede a sus miembros con el operador flecha (->) en vez del punto (.).
- ▶ '(*p).metodo()' es equivalente a 'p->metodo()', pero la segunda forma es la convención estándar.
- ▶ Todo new debe tener su delete correspondiente; cada delete debe ejecutarse exactamente una vez por objeto.

main.cpp

```
Libro* p = new Libro("El Quijote", "Cervantes");

p->prestar();           // forma estándar
(*p).devolver();       // equivalente, poco usado

cout << p->getTitulo();

delete p;              // libera el heap
p = nullptr;          // evita puntero colgante
```

 **Error frecuente:** Usar un puntero después de hacer delete sobre él (puntero colgante / dangling pointer) sin ponerlo en nullptr.

? ¿Qué pasaría si accedo a `p->getTitulo()` después de `delete p`; sin haber puesto `p = nullptr`;

Vector dinámico (paso 1): crear y agregar

- ▶ Aplicamos al catálogo de la Biblioteca la estrategia directa vista en el Módulo 0: sin variable de capacidad, sólo 'cantidad'.
- ▶ Crear: arrancar con un arreglo vacío (`new Libro*[0]`).
- ▶ Agregar: crear un arreglo de tamaño `cantidad+1`, copiar los punteros existentes, sumar el nuevo `Libro*` al final y liberar el arreglo viejo.
- ▶ Cada `agregarLibro()` reserva memoria nueva del tamaño EXACTO necesario: es la versión más simple, aunque copia todo en cada inserción.

vector dinámico — crear/agregar

```
Libro** catalogo = new Libro*[0];
int cantidad = 0;

void agregarLibro(Libro* nuevoLibro) {
    Libro** nuevo = new Libro*[cantidad + 1];
    for (int i = 0; i < cantidad; i++)
        nuevo[i] = catalogo[i];
    nuevo[cantidad] = nuevoLibro;
    delete[] catalogo;    // libera el arreglo viejo
    catalogo = nuevo;
    cantidad++;
}
```

Memoria antes/después de agregar

```
ANTES: [Rayuela][Quijote]  cantidad=2
agregarLibro(...)
DESPUÉS: [Rayuela][Quijote][CienAnios]  cantidad=3
(arreglo NUEVO de 3)
```

? ¿Qué diferencia hay entre esta estrategia y reservar de a bloques (por ejemplo, de a 4 en 4)? ¿Qué se gana y qué se pierde con cada una?

Vector dinámico del catálogo: eliminar, copiar y liberar

- ▶ Eliminar un libro: crear un arreglo de tamaño cantidad-1, copiar todos los punteros salvo el eliminado, y liberar el arreglo viejo.
- ▶ El objeto Libro señalado por la posición eliminada debe liberarse con delete ANTES de descartar su puntero.
- ▶ Copiar el catálogo completo implica copiar cada objeto apuntado (deep copy) si se desea independencia total — se retoma en Módulo 7.
- ▶ Liberar todo: delete de cada objeto individual y luego delete[] del arreglo de punteros.

redimensionar y liberar

```
void eliminarLibro(int indice) {
    delete catalogo[indice]; // libera el objeto
    Libro** nuevo = new Libro*[cantidad - 1];
    int j = 0;
    for (int i = 0; i < cantidad; i++)
        if (i != indice) nuevo[j++] = catalogo[i];
    delete[] catalogo;
    catalogo = nuevo;
    cantidad--;
}

// liberar todo al finalizar
for (int i=0;i<cantidad;i++) delete catalogo[i];
delete[] catalogo;
```

 **Error frecuente:** Liberar el arreglo de punteros (delete[] catalogo;) sin antes hacer delete de cada Libro* individual: los objetos Libro quedan como memory leaks.

 ¿Por qué eliminarLibro() necesita DOS delete distintos: uno para el objeto y otro (delete[]) para el arreglo?

Resumen — Memoria Dinámica y Vectores Dinámicos

Bloom: Aplicar

 5 min

✓ Ideas clave

- Stack se libera automáticamente; heap requiere delete manual.
- delete[] para arreglos, delete para objetos individuales; cada new necesita exactamente un delete.
- Vector dinámico (estrategia directa): crear, agregar, eliminar, copiar y liberar son las cinco operaciones clave.
- Agregar o eliminar implica reservar un arreglo del tamaño exacto necesario, copiar los punteros existentes y liberar el arreglo viejo.
- Reservar memoria de a bloques reduce la cantidad de veces que debemos copiar todo el vector, a costa de mantener algunas posiciones libres temporalmente.

⚠ Errores frecuentes

- Memory leaks: new sin delete correspondiente.
- delete simple sobre un arreglo (debe ser delete[]).
- Punteros colgantes: usar un puntero después de delete sin ponerlo en nullptr.
- Liberar el arreglo de punteros sin antes liberar cada objeto individual (o al revés, liberar el objeto y dejar el puntero colgando en el arreglo nuevo).

📌 Ejercicio rápido

Escribí la función eliminar(int indice) para el vector dinámico de libros, que borre el objeto en esa posición y desplace los siguientes para no dejar huecos.

MÓDULO

7

≈ 20 minutos

Copia de Objetos

Objetivos de esta sección

- Explicar el funcionamiento del constructor de copia y el operador de asignación.
- Diferenciar shallow copy de deep copy.
- Aplicar la Regla de los Tres para evitar double delete.

Constructor de copia y operador de asignación

- ▶ **Constructor de copia:** se invoca al crear un objeto nuevo a partir de otro existente (`Libro l2 = l1;` o `Libro l2(l1);`).
- ▶ **Operador de asignación (`operator=`):** se invoca cuando ambos objetos ya existen y se asigna uno a otro (`l2 = l1;`).
- ▶ Si no se definen, el compilador genera versiones por defecto que copian miembro a miembro (**shallow copy**).
- ▶ El shallow copy es peligroso cuando la clase gestiona memoria dinámica: ambos objetos terminan apuntando al mismo bloque de heap.

firmas típicas

```
class Libro {
    char* titulo;    // memoria dinámica
public:
    Libro(const Libro& otro);           // copia
    Libro& operator=(const Libro& otro); // asign.
    ~Libro();
};

Libro l1("Rayuela");
Libro l2 = l1;    // constructor de copia: Aquí l2 no
                  existía.
Libro l3("X");
l3 = l1;          // operador de asignación: Si el
                  objeto ya existía, entonces sí es una asignación y se
                  llama a operator=.
```

? ¿En qué se diferencia '`Libro l2 = l1;`' (declaración) de '`l3 = l1;`' (`l3` ya existía)?

Shallow copy vs. Deep copy, y la Regla de los Tres

- ▶ **Shallow copy:** copia los punteros tal cual; ambos objetos terminan apuntando al mismo bloque de memoria.
- ▶ **Deep copy:** reserva memoria nueva e independiente, copiando el contenido apuntado, no la dirección.
- ▶ **Regla de los Tres:** si una clase define manualmente destructor, constructor de copia u operador de asignación, probablemente necesite definir los tres.
- ▶ La razón: los tres están relacionados con la gestión de recursos propios (memoria dinámica).

Shallow vs. Deep copy

```
SHALLOW COPY          DEEP COPY
11.titulo --+          11.titulo --> ["Rayuela"]
               v
               ["Rayuela"]      12.titulo --> ["Rayuela"]
(copia)
12.titulo --+
(mismo bloque = riesgo de double delete)
```

⚠ **Error frecuente:** Definir el destructor (para liberar memoria) pero olvidar el constructor de copia y operador=, dejando la copia por defecto (shallow).

? Si una clase sólo tiene atributos simples (int, double, bool) sin punteros, ¿necesita aplicar la Regla de los Tres?

Double Delete: el bug y su corrección

- ▶ Sin constructor de copia propio, 'Libro l2 = l1;' copia el puntero titulo, no la cadena.
- ▶ Al destruirse l1 y l2, AMBOS destructores intentan liberar la MISMA dirección de memoria: double delete (comportamiento indefinido, suele crashear).
- ▶ La corrección reserva memoria nueva en el constructor de copia y en operator=, copiando el contenido (deep copy).

✗ CON BUG (shallow, x defecto)

```
class Libro {
    char* titulo;
public:
    Libro(const char* t) {
        titulo=new char[strlen(t)+1];
        strcpy(titulo, t); }
    ~Libro(){ delete[] titulo; }
    // sin ctor de copia propio:
    // se copia el PUNTERO
}; // l2=l1 -> doble delete
```

✓ CORREGIDO (deep copy)

```
Libro(const Libro& otro) {
    titulo = new char[
        strlen(otro.titulo)+1];
    strcpy(titulo,
        otro.titulo);
}
// memoria propia e
// independiente por objeto
```

? ¿Por qué el operador de asignación también necesita liberar su propia memoria ANTES de copiar la nueva (y verificar auto-asignación)?

Resumen — Copia de Objetos

✓ Ideas clave

- **Constructor de copia:** objeto nuevo a partir de otro existente. Operador=: ambos objetos ya existían.
- **Shallow copy** copia punteros;
- **deep copy** copia el contenido apuntado con memoria propia.
- Regla de los Tres: destructor, constructor de copia y operator= suelen ir juntos cuando hay memoria dinámica.
- El double delete es consecuencia típica de una shallow copy no intencional sobre recursos dinámicos.

⚠ Errores frecuentes

- Confiar en el constructor de copia por defecto cuando la clase gestiona memoria dinámica.
- Definir destructor sin definir constructor de copia ni operator= (viola la Regla de los Tres).
- Olvidar liberar la memoria previa dentro de operator= antes de copiar la nueva.
- No verificar auto-asignación (`l1 = l1;`) en operator=.

📝 Ejercicio rápido

Escribí el operador de asignación (operator=) para la clase Libro con deep copy, incluyendo la verificación de auto-asignación (`this != &otro`).